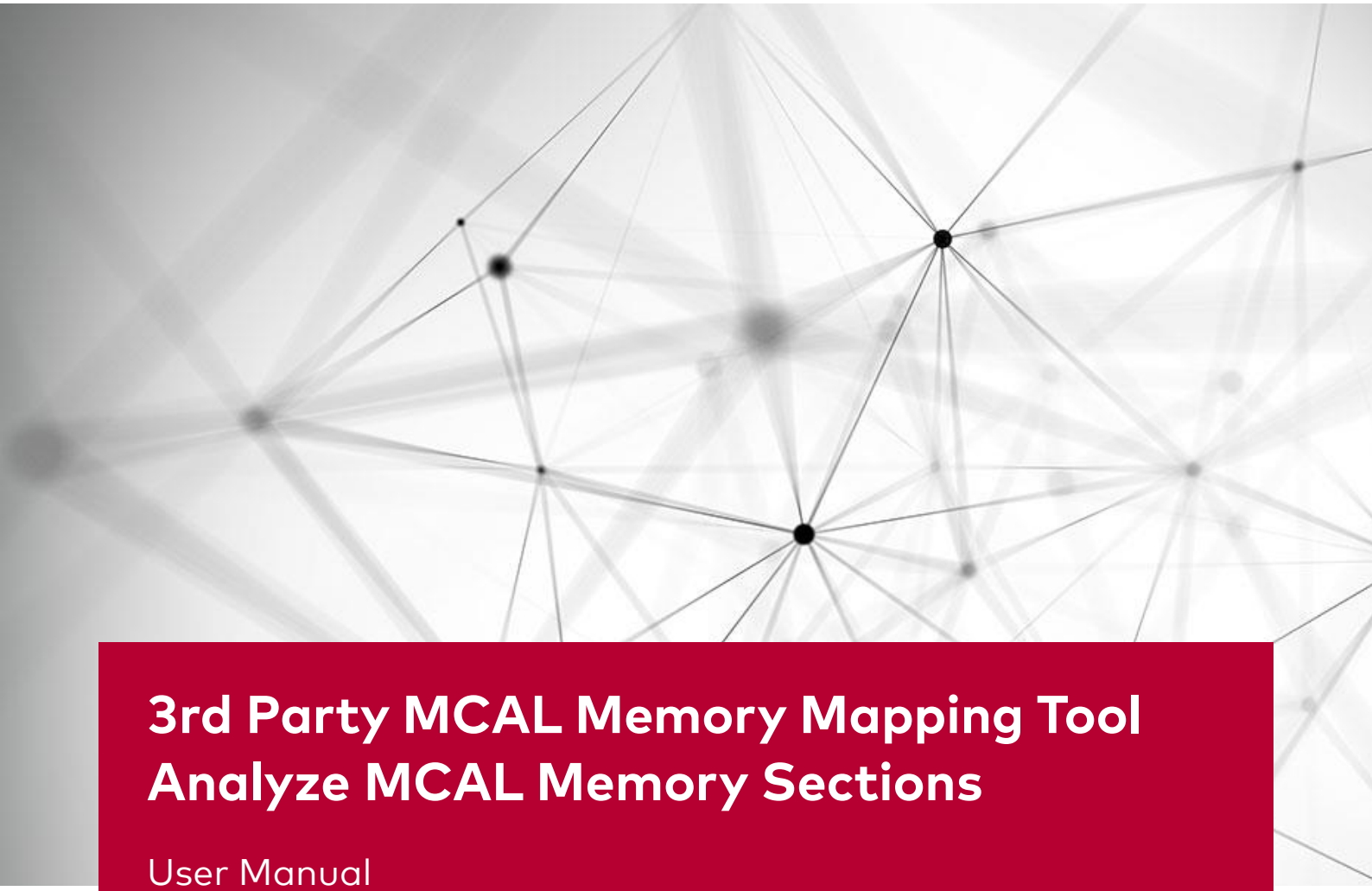
VECTOR



3rd Party MCAL Memory Mapping Tool Analyze MCAL Memory Sections

User Manual

Lenka Gerlach

V1.0.0 | Released

Document Information

History

Author	Date	Version	Remarks
virld	2022-04-28	1.0.0	Create initial version



Caution
This document gives an overview on product-wide behavior or use-cases. To ensure correct operation of the product always follow the instructions provided in the technical references and (additionally for safety projects) the safety manual. The content of these documents always replaces/overrides the content from this user manual.

Contents

1 Purpose of the Document 5

2 Introduction 6

3 Tool Interface 7
3.1 Main Window 7
3.2 Menu Strip 8
3.3 Lookup Page 9
3.4 Mapper Page 9

4 Memory Mapping – Step by Step 11
4.1 Step 1 - Choose MCAL Package 11
4.2 Step 2 - View Search Results 12
4.3 Step 3 - Choose Modules 13
4.4 Step 4 - Memory Mapping 14
4.5 Step 5 - Merging Files 15
4.5.1 Merging Files Automatically 15
4.5.2 Compare Files With an External Tool 17

5 Find Errors 19
5.1 Errors and Warnings 20
5.1.1 Errors 20
5.1.2 Warnings 21

6 Compiler Configuration Analysis 23

7 Glossary and Abbreviations 25
7.1 Glossary 25
7.2 Abbreviations 25

8 Contact 26

Illustrations

Figure 3-1 Main Window With First Page 7
Figure 3-2 Menu Strip 8
Figure 3-3 Options Dialog 8
Figure 3-4 Lookup Page 9
Figure 3-5 Mapper Page 10
Figure 4-1 MCAL Package Path Selection 11
Figure 4-2 Search Result Memory Sections 12
Figure 4-3 Module Selection 13
Figure 4-4 Memory Section Mappings 14
Figure 4-5 File Selection in Merge Dialog 16
Figure 4-6 Merged Sections 16
Figure 4-7 Set Path to the Compare Tool in Options Dialog 17
Figure 4-8 Select Files for Compare 18
Figure 5-1 Message in Module Errors Tab 19
Figure 5-2 Filter for Errors and Warnings 20
Figure 6-1 Compiler Configuration File Analysis 23
Figure 6-2 Compiler Flag Configuration in Options Dialog 24

Tables

Table 4-1 Buttons on the Lookup Page 13
Table 4-2 Buttons on the Mapper Page 15
Table 5-1 Description of Errors 21
Table 5-2 Description of Warnings 22

1 Purpose of the Document

The goal of this document is to provide an overview of the 3rd Party MCAL Memory Mapping tool, hereafter referred to as **3rdPartyMcalMemMap** tool. Basic functionalities and features are presented in the following chapters.

The document is divided into two parts: the first part presents the theoretical part with information about the tool structure and its features. The second part, starting from chapter 4, contains an instruction that you can follow to find memory sections or perform a mapping, for example.

2 Introduction

The 3rdPartyMcalMemMap tool is a memory mapping tool used mainly for the assignment of memory sections and the search for exclusive areas in an MCAL package. This process can be very tedious when performed manually. Therefore, tool support simplifies this task by analyzing an MCAL package according to following aspects:

- Find all memory sections in an MCAL package.
- Assign these memory sections to Vector specific memory sections.
- Create the output file `Mcal_MemMap.inc` which contains all assignments. The file has template character, which means that it contains mapping proposals.
- Find exclusive areas in the MCAL package.
- Check for possible errors regarding memory section definitions in the MCAL sources.

For example, the following possible errors are identified:

- Check if `#include "MemMap.h"` is missing after any START or STOP memory section.
- Check if all directive constructs are closed.
- Check if there is a matching STOP memory section for every START memory section.
- Check if there is a matching START memory section for every STOP memory section.

© 2022 Vector Informatik GmbH

3 Tool Interface

This chapter describes the graphical user interface of the tool such as the structure of the main window, its menu bar, its buttons and its different pages.

3.1 Main Window

The tool is designed like a wizard that guides you through the various steps of creating and managing a memory mapping file. The first page deals with memory section and exclusive area lookup while the second page offers mainly functionality for creating, comparing and merging memory mapping files.

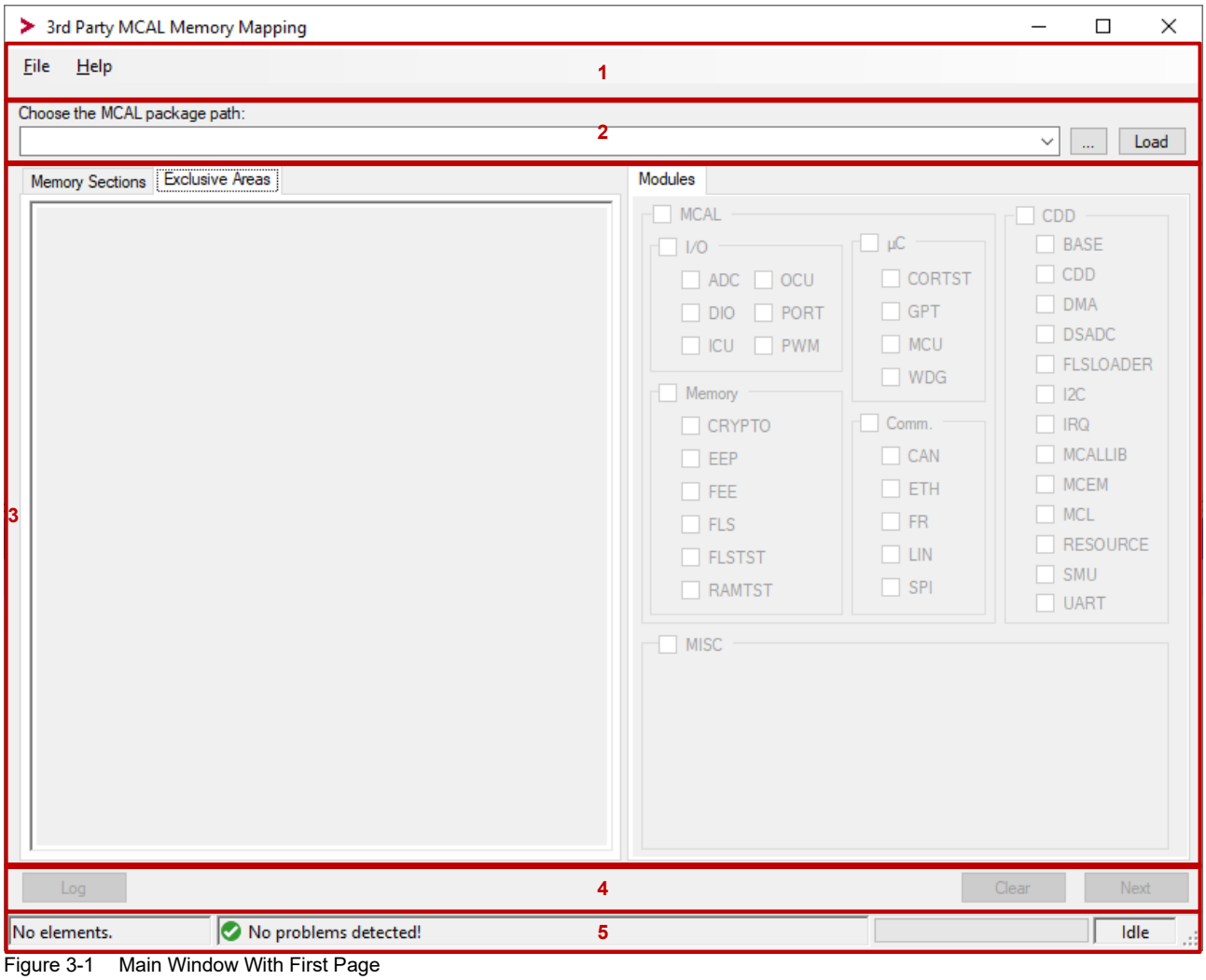


Figure 3-1 Main Window With First Page

As seen in the picture, the application consists of several sections:

> The **menu strip** (1) provides basic application features like settings or help.

> The **path selector** (2) is used for specifying source or target paths for each page.

- The **wizard page panel** (3) displays the search results such as memory sections found.
- The **control panel** (4) contains the page controls such as the **[Next]** button to go to the second page.
- The **status bar** (5) contains an elements count, a progress bar indicating the current operation progress e.g. during the search and an area for the application's status.

3.2 Menu Strip

The menu strip offers two dropdown menus. In the screenshots below you can see the menu items including their shortcuts, if available.



Figure 3-2 Menu Strip

- **File** | **Options** will bring up the following window where you can change the default paths to the editor and the compare tool executables, and the default save paths of the log and the `Mcal_MemMap.inc` file.

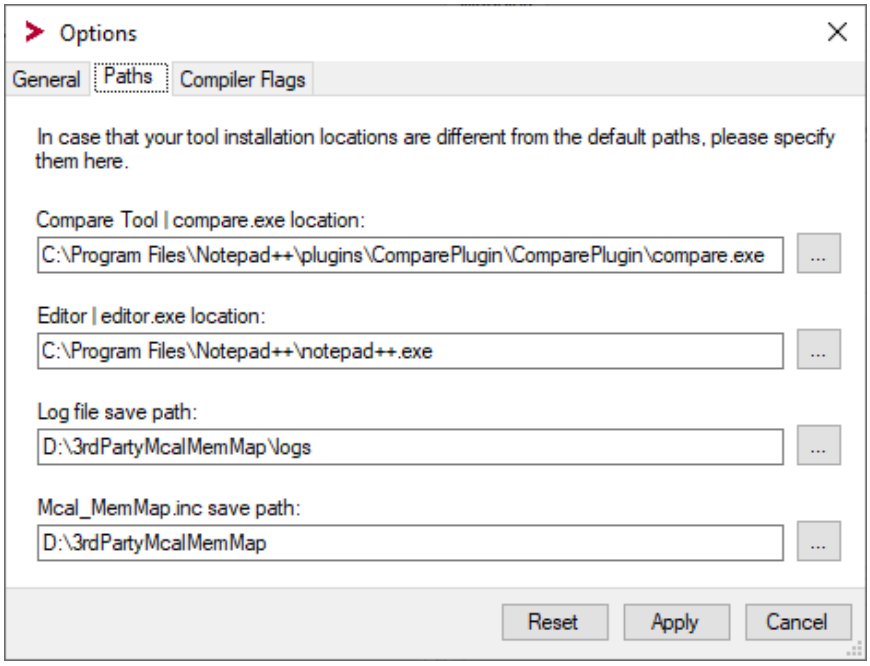


Figure 3-3 Options Dialog

- **File** | **Exit** will exit the application.
- **Help** | **Help** opens this user manual, if it is in the correct directory which means that this file should be in the same folder as the `MCALMemMap.exe` file.
- **Help** | **About** shows information about the tool such as the tool version.

3.3 Lookup Page

The first page of the application is used for memory section and exclusive area lookup. It includes the **path entry control** where you specify the MCAL package which will be searched for memory sections and exclusive areas. The search results are shown in the tabs **Memory Sections** and **Exclusive Areas** and the associated modules are displayed in the **Modules** tab:

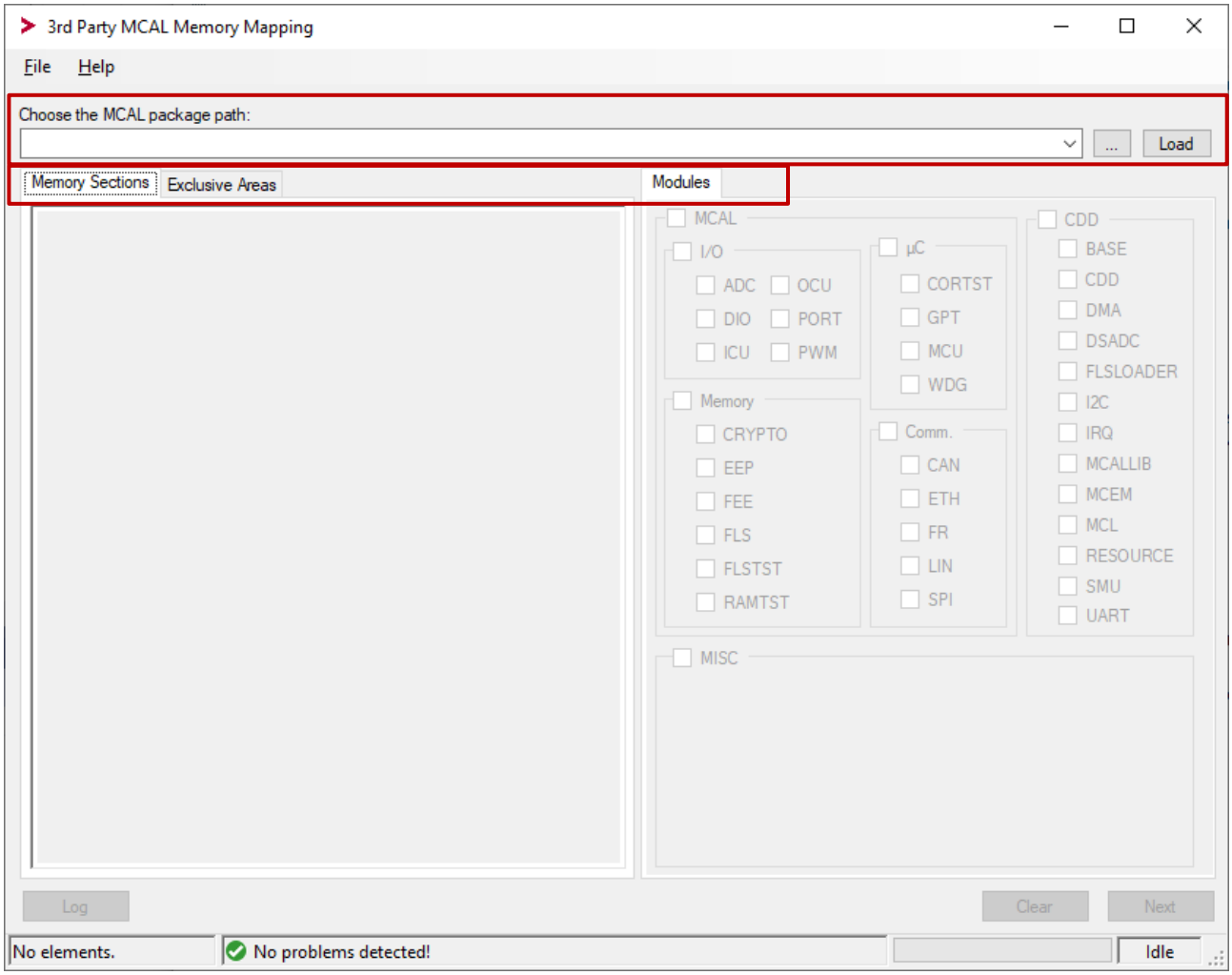


Figure 3-4 Lookup Page

The **Memory Sections** tab will display any `START_SEC` defines found during lookup and the **Exclusive Areas** tab any `SchM_Enter` sections. The displayed entries can be filtered via the **Modules** panel by checking or unchecking the according checkbox.

3.4 Mapper Page

The second page is mainly used for mapping the previously looked up sections into an `Mcal_MemMap.inc` file. It also provides the possibility to merge missing sections from one mapping file into another. Each operation will display its results in the matching tab on the left, while the **Progress** panel on the right displays if the single operation steps were finished successfully or not:

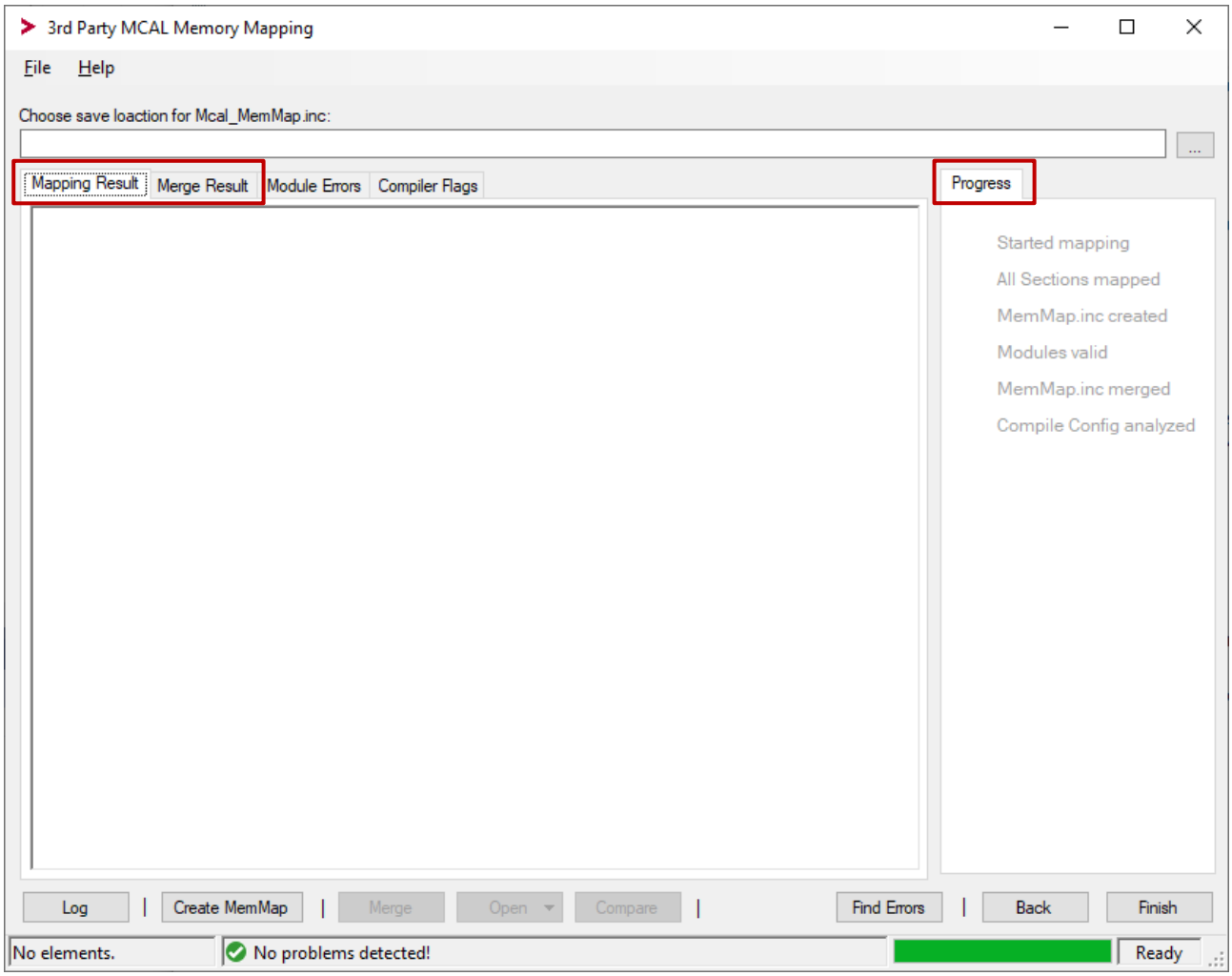


Figure 3-5 Mapper Page

4 Memory Mapping – Step by Step

This chapter describes in detail the steps starting from the selection of the MCAL package, its analysis, the creation of the memory mappings up to the comparison of two different memory mapping files.

4.1 Step 1 - Choose MCAL Package

The first step is to choose a path, which needs to point at your MCAL package, that you wish to search through. You can either copy & paste your path into the text field and press **Enter** or click the **[...]** button and navigate through the folder dialog:

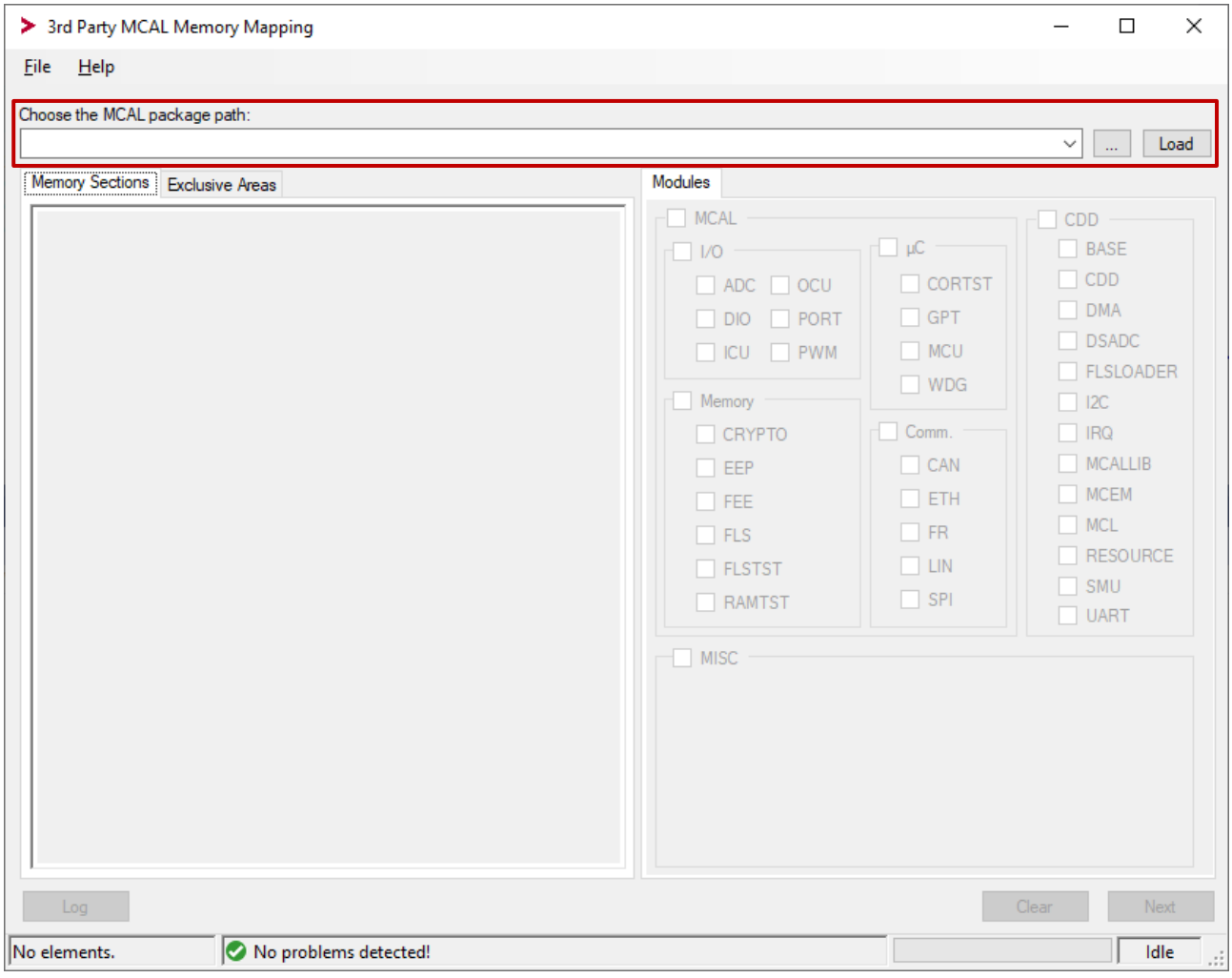


Figure 4-1 MCAL Package Path Selection

If you have already run the tool before and searched for an MCAL package, you can choose the previous MCAL package from the dropdown menu. Hit **[Load]** or alternatively the **Enter** key and move to the next step.

4.2 Step 2 - View Search Results

Once you have chosen your path, the tool will search given directory for `*.c` and `*.h` files. It will ignore files such as `MemMap.h`, `Mem_Map.h`, `Adc_Mem_Map.h` and similar. The results will then be displayed in the left panel. Tab **Memory Sections** shows all found memory sections and tab **Exclusive Areas** shows all found exclusive areas:

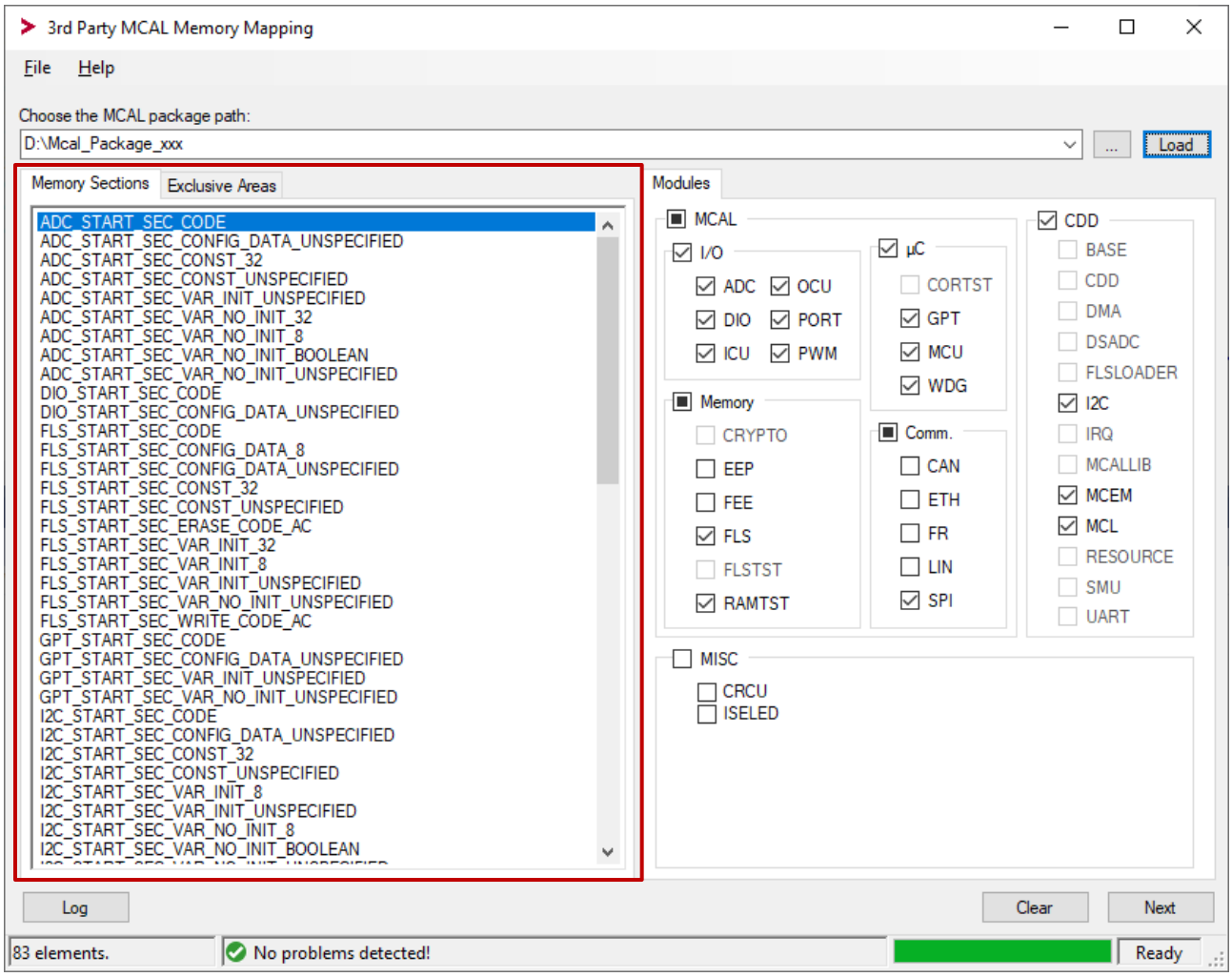


Figure 4-2 Search Result Memory Sections

Within the tab content you have the option to mark

- a single line,
- all the lines with <CTRL>+<A>,
- a range of lines with <SHIFT>+<Left click>,
- a variety of lines with <ALT>+<Left click>.

You can copy the selected lines to clipboard via <CTRL>+<C>.

Below the panel there are several buttons available. The use of the buttons is described in following table:

Button	Action
[Log]	Opens the current log file, if available
[Clear]	Clears all the data, as if you relaunch the application. Choose a new path afterwards and hit **[Load]** to see the results of this MCAL package.
[Next]	Opens next page

Table 4-1 Buttons on the Lookup Page

4.3 Step 3 - Choose Modules

On the right side of the lookup page you can see the **Modules** tab. It displays the modules recognized during lookup:

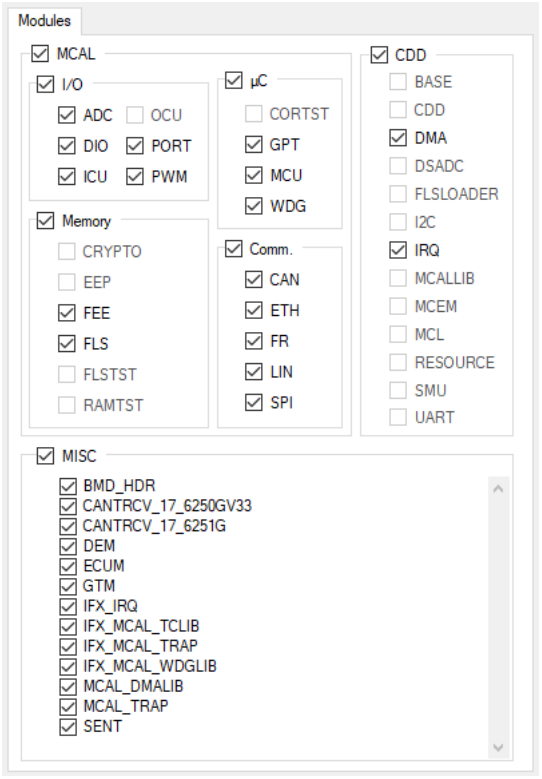


Figure 4-3 Module Selection

While **MCAL** and **CDD** modules are predefined, **MISC** modules are added on the fly during the lookup of the MCAL package. These MISC modules are all modules that are found in the MCAL package and that are neither an MCAL nor a CDD module.

If you want to exclude any module or module group, uncheck the respective checkbox and any changes in the memory sections and exclusive areas will be applied immediately. Please note, that excluding a module will lead to ignoring that module in any of the following operations.

After you are done selecting your modules, hit **[Next]** in the control bar and move on to the next step.

4.4 Step 4 - Memory Mapping

The second page of the application is the mapping page. This step involves the assignment of the MCAL memory sections to Vector specific memory sections. They will be created and saved to an `Mcal_MemMap.inc` file.

You have the option to set the save path for the `Mcal_MemMap.inc` file in the upper part of the page by pressing the **[...]** button and navigating to the save directory of your choice. Alternatively, you can enter the path directly into the field.

To start the creation of the sections, press the **[Create MemMap]** button. Be aware that this could take a little bit of time, depending on the amount and complexity of the memory sections. After the memory sections have been mapped and the `Mcal_MemMap.inc` file has been created, the content of the **Mapping Result** tab shows these mappings:

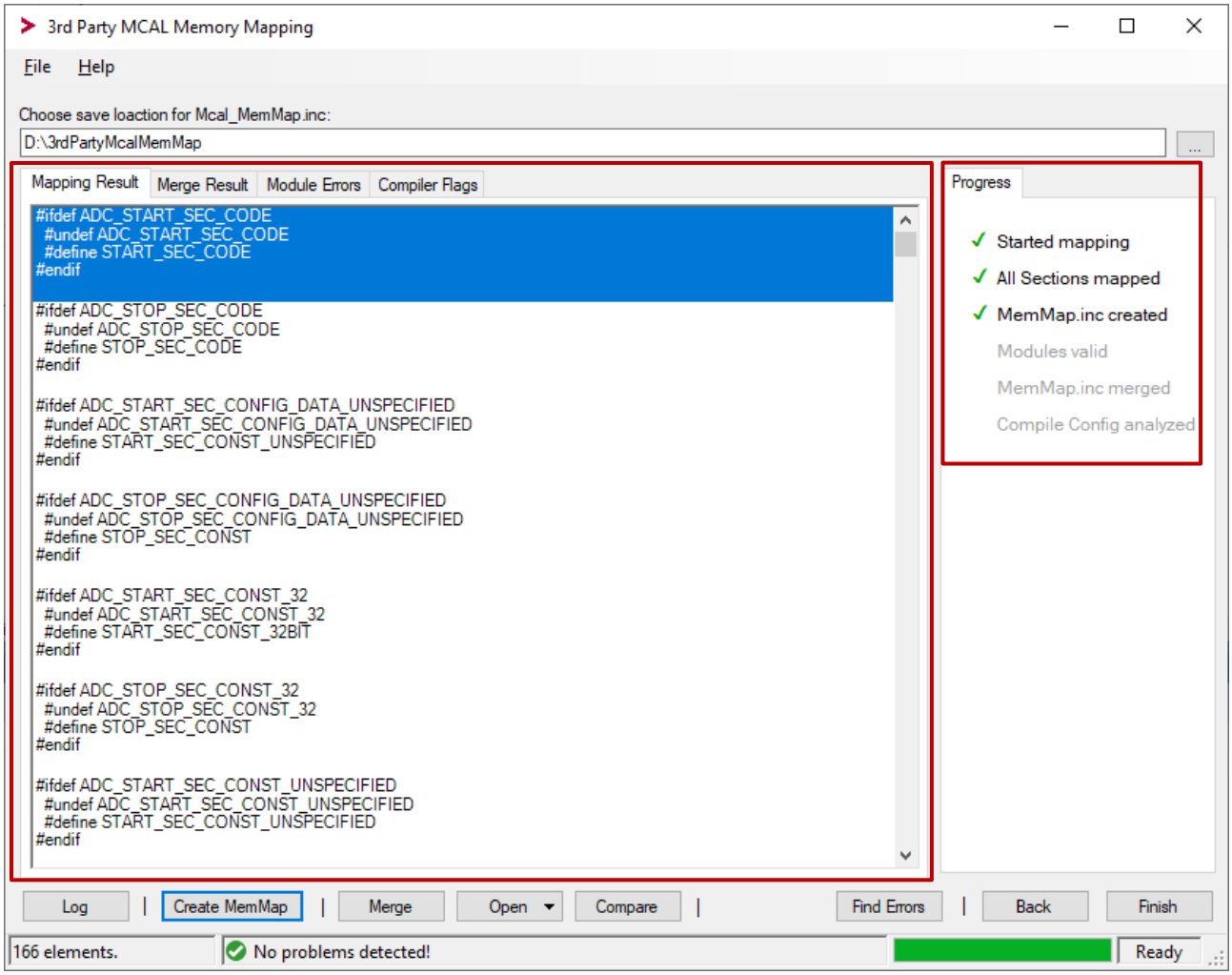


Figure 4-4 Memory Section Mappings

You can select the mappings and copy them to the clipboard.

On the right side is the **Progress** tab. It shows the single steps of the mapping process and marks them if errors have been encountered or if they have been successfully finished. In the case of an error, a red X will be displayed instead of the green check mark.

You can leave this page and go back any time to the previous page in case you want to reconfigure your selected modules by pressing the **[Back]** button.

The buttons **[Merge]**, **[Open]** and **[Compare]** are only enabled after the mapping file is created.

The following table contains a brief description of all buttons while the next chapters go into more detail:

Button	Action
[Log]	Opens the current log file
[Create MemMap]	Maps the memory sections and creates the Mcal_MemMap.inc file
[Merge]	Automatically merges the missing sections from the generated mapping file into a selected one
[Open]	Opens the just created Mcal_MemMap.inc file
[Compare]	Alternative to **[Merge]** if you want to merge manually (requires external compare tool)
[Find Errors]	Starts the searching for errors
[Back]	Opens the first step of the tool again
[Finish]	Closes the application

Table 4-2 Buttons on the Mapper Page

4.5 Step 5 - Merging Files

To trigger a merge, you must have previously created a `Mcal_MemMap.inc` file via **[Create MemMap]**. If you did not, go back to 4.4 and continue from there. Otherwise, you can trigger an automatic or a manual merge. Both methods are described in the next chapters.

4.5.1 Merging Files Automatically

Via the **[Merge]** button you start the automatically insert of all missing memory sections from your generated file into a selected file. Doing this will first open the following dialog window where you can choose an `*.inc` file to merge into and specify whether to overwrite the chosen file or to create a new one next to it.

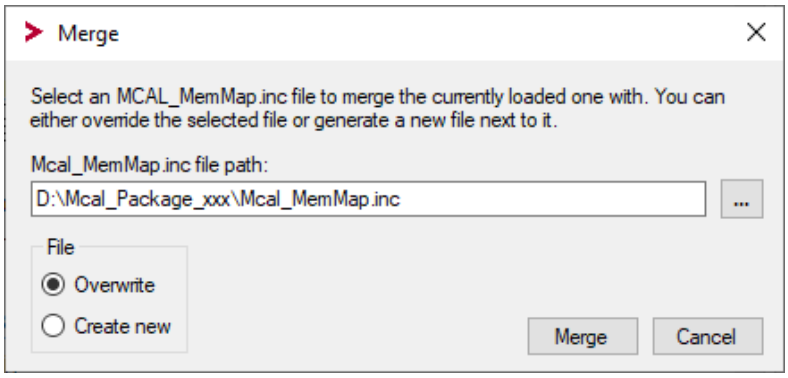


Figure 4-5 File Selection in Merge Dialog

If merging finishes successfully, an info box will pop up telling you the number of inserted sections and the generated results will be displayed in the **Merge Result** tab:

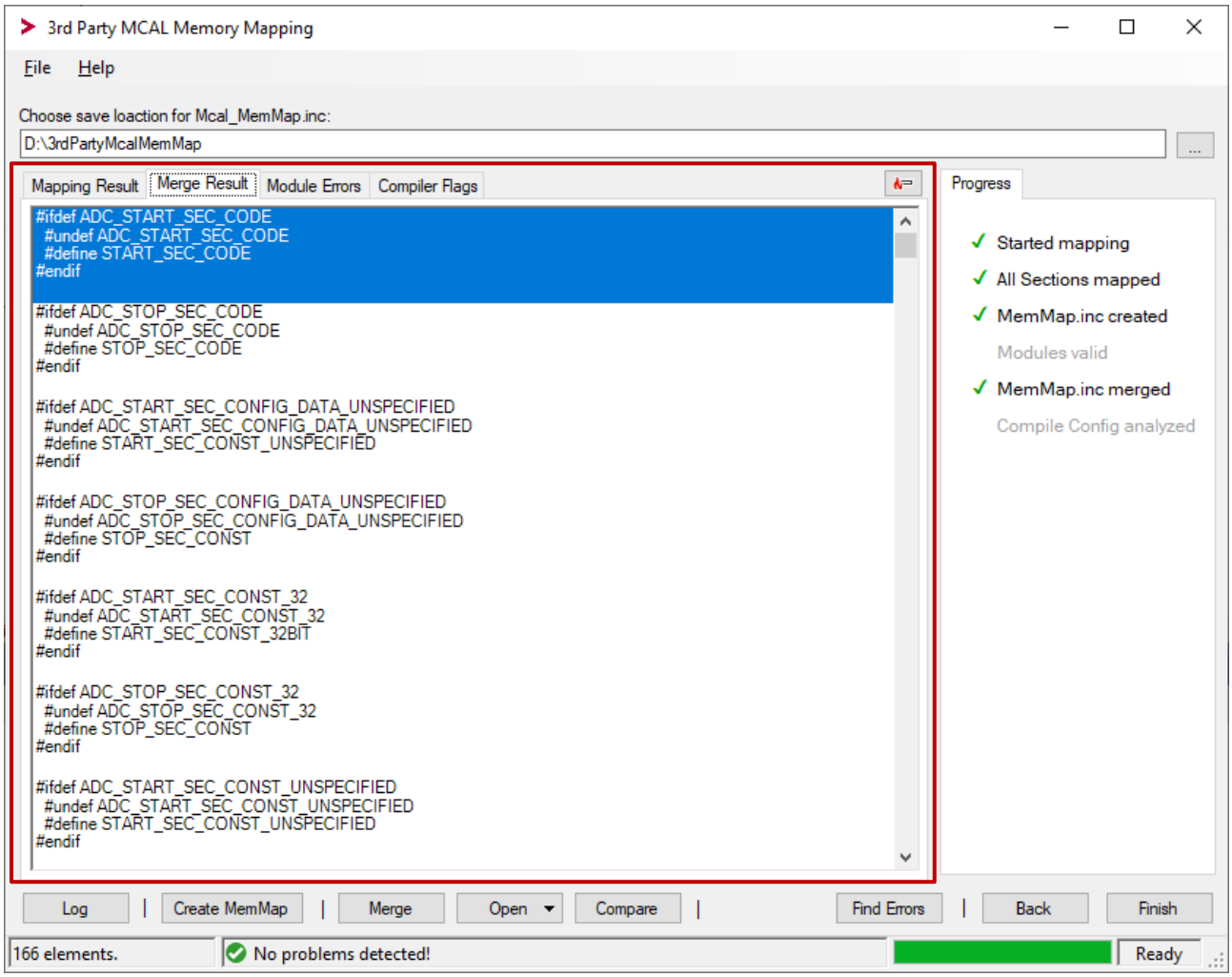


Figure 4-6 Merged Sections

Note
All mappings in the file in which you merged into is sorted alphabetically and then saved.

The button in the upper right corner of the **Merge Result** tab toggles between the complete merge result or the merged sections only.

4.5.2 Compare Files With an External Tool

As an alternative to automatic merging, you can manually compare and merge via the **[Compare]** button.

Before you can use this comparison option you need to have an external compare tool like **Notepad++** or **Beyond Compare** installed. After installation the path to the executable needs to be set within the 3rdPartyMcalMemMap tool options. Press F1 or go to **File** | **Options** and then switch to the **Paths** tab:

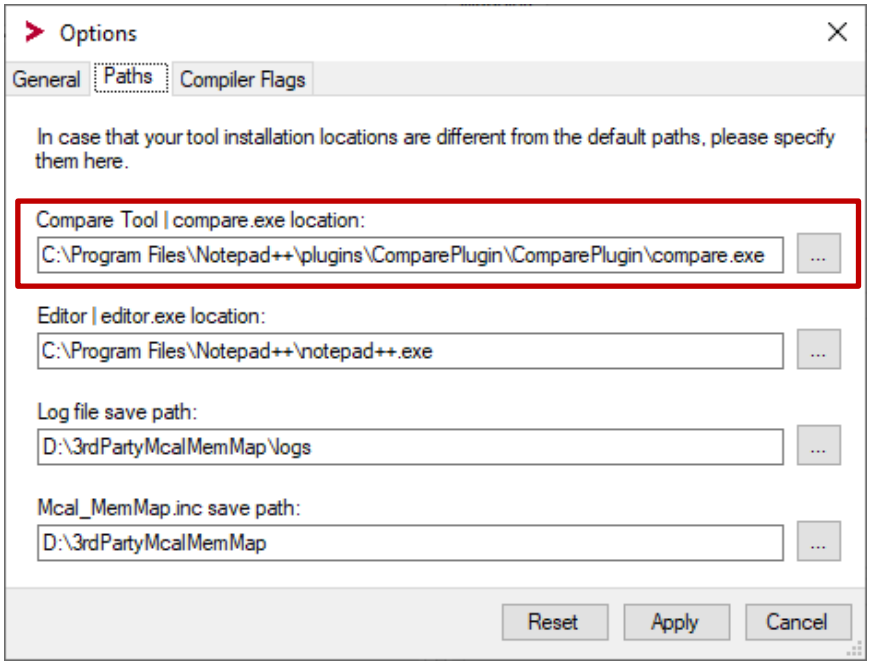


Figure 4-7 Set Path to the Compare Tool in Options Dialog

Here you can find and select the compare executable within your folder structure, in case that it is different from the given default paths. After you're done, click **[Apply]** afterwards.

Back to the mapping page, pressing the **[Compare]** button will open the following window:

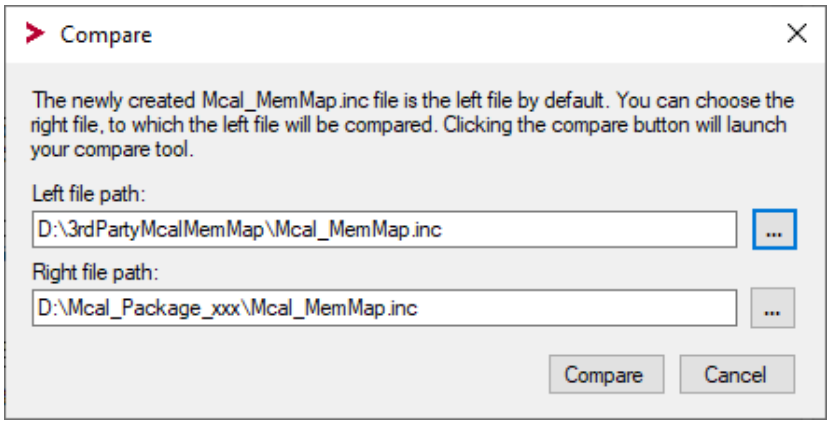


Figure 4-8 Select Files for Compare

In this dialog you can select the two files you want to compare. The **Left file** in this case is defaulted to the `Mcal_MemMap.inc` file that you have created before. You can choose to change the left file if you wish to.

The **Right file** is the file that you wish to compare to your left file. Paste the path into the text field or use the **[...]** button to navigate to the file of your choice. Once ready, press the **[Compare]** button, and the configured compare application should launch with your files. Now you can proceed in this application with a manual merge of these files.

5 Find Errors

This feature supports the user in finding common errors in the MCAL package files regarding memory sections such as missing `MemMap.h` inclusions and unopened or unclosed memory sections.

Pressing the **[Find Errors]** button will initiate the error searching feature. Depending on the MCAL package, this can take a bit of time. The results are then displayed in the **Module Errors** tab. If no errors were found, the error list will display an entry stating that and the label in the **Progress** panel will state a successful execution.

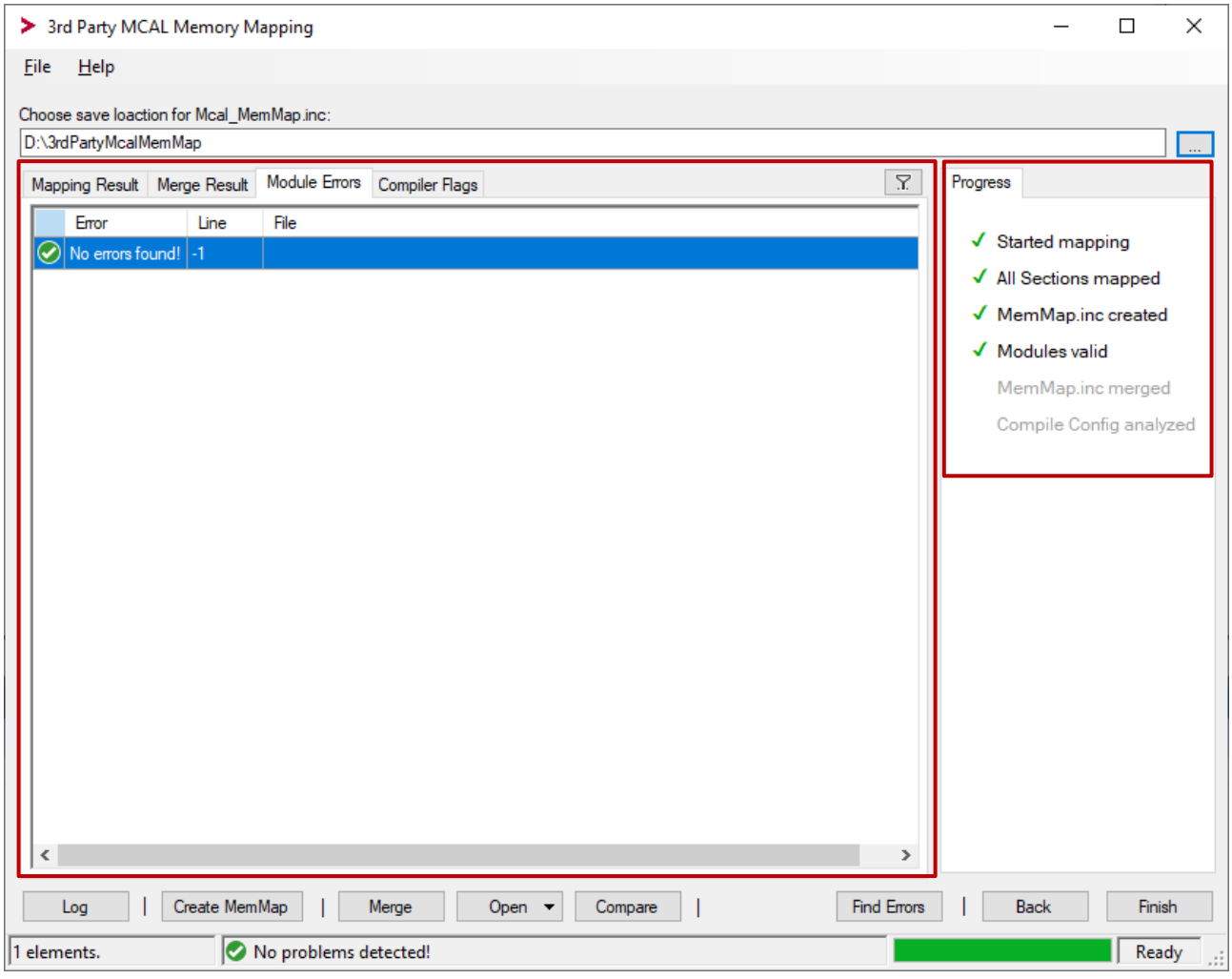


Figure 5-1 Message in Module Errors Tab

If errors are found, then

- First column contains an icon that shows if there is an error or a warning or if there are no errors and warnings.
- Error column contains a short description of each error.
- Line column shows the line in the file where the error occurred.

© 2022 Vector Informatik GmbH

> File column contains the affected file path. Double-click the file path to open the file in your default file editor. You can change the file editor to Notepad++ editor in File | Options | Paths. The use of this tool allows not only to open the file but also to jump to the error line.

The error list can be filtered by error type upon clicking the filter button in the top right corner. Deselecting a checkbox or group will exclude the errors from being displayed, while still being tested during lookup. Thus, the filtering is a tool for giving a better overview when many errors are found rather than improving error lookup performance.

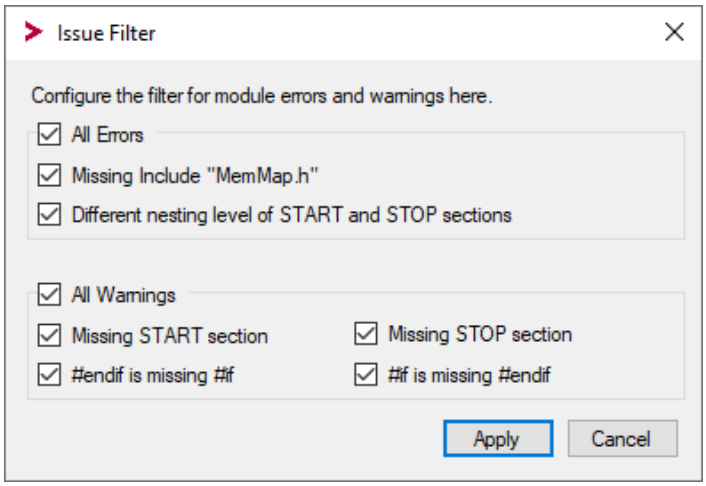


Figure 5-2 Filter for Errors and Warnings

5.1 Errors and Warnings

The following two tables contain all problems that are detected. The first table contains all errors, and the second table contains all warnings.

5.1.1 Errors

Text	Cause	Line number references …
Missing include "MemMap.h"	Between two memory section defines, no `#include "MemMap.h"` (or any variant) is found	…the #define declaring the memory section missing the #include
Different nesting level of START and STOP section	There is a START section wrapped inside an `#if` directive whereas the STOP section is outside of it. Thus, the START section might happen to not be defined while the STOP section will be. Example:	…the START section definition wrapped inside an #IF directive

	`#if (GPT_FTM_USED == STD_ON)`
	`#define GPT_START_SEC_CODE`
	`#include "Gpt_MemMap.h"`
	`#endif /*GPT_FTM_USED == STD_ON*/`
	`#define GPT_STOP_SEC_CODE`
	`#include "Gpt_MemMap.h"`	

First section must be START section	First memory section in a file must be a START section	…the first section definition
Last section must be STOP section	Last memory section in a file must be a START section	…the last section definition

START section is not followed by its STOP section	Indicates that two or more START sections are defined without being closed before by their STOP sections.
	Example:
	`#define R_FDL_START_SEC_CONST`
	`#include "r_fdl_mem_map.h"`
	`#define R_FDL_START_SEC_PUBLIC_CODE`
	`#include "r_fdl_mem_map.h"`
		…the first START section definition

Table 5-1 Description of Errors

5.1.2 Warnings

Text	Cause	Line number references …
#if is missing	Not every `#endif` directive has an opening `#if` directive	…the last #endif directive that has no matching #if
#endif is missing	Not every `#if` directive has a closing `#endif` directive	…the last #if directive that has no matching #endif
Missing START section	There is a STOP section defined without a matching START section	…the #define declaring the STOP section missing a START section
Missing STOP section	There is a START section defined without a matching STOP section	…the #define declaring the START section missing a STOP section
Api Infix may be missing	`SchM_Enter` function, which enters an exclusive area, contains a vendor id but no vendor Api infix. Examples: Vendor Api infix is **not** missing: `SchM_Enter_Wdg_30_Inst1_WDG_RAM_PROTECT` Vendor Api infix **is** missing:	…the affected exclusive area definition

	`SchM_Enter_Wdg_30_WDG_RAM_PROTECT //Inst1 misses`
	
	Note:
	The algorithm to find this error currently cannot distinguish between this error and exclusive area names which **do not start with their module name** such as:
	`SchM_Enter_Wdg_RAM_PROTECT(void) //WDG misses`
	
	Therefore, this is only a warning for checking if Api infix is really missing or not.	

Exclusive area creation not possible	`SchM_Enter` function name from which no exclusive area name can be derived.
	Example:
	`#define SchM_Enter_Wdg_(ExclusiveArea)`
	`SchM_Enter_Wdg_##ExclusiveArea`
		…the affected exclusive area definition

Table 5-2 Description of Warnings

6 Compiler Configuration Analysis

The 3rdPartyMcalMemMap tool offers the possibility to check a compiler configuration file usually called `Compiler_Cfg.h` for missing entries. Therefore, you switch on the mapper page to the tab **Compiler Flags** and via drag & drop the compiler configuration file can be added:

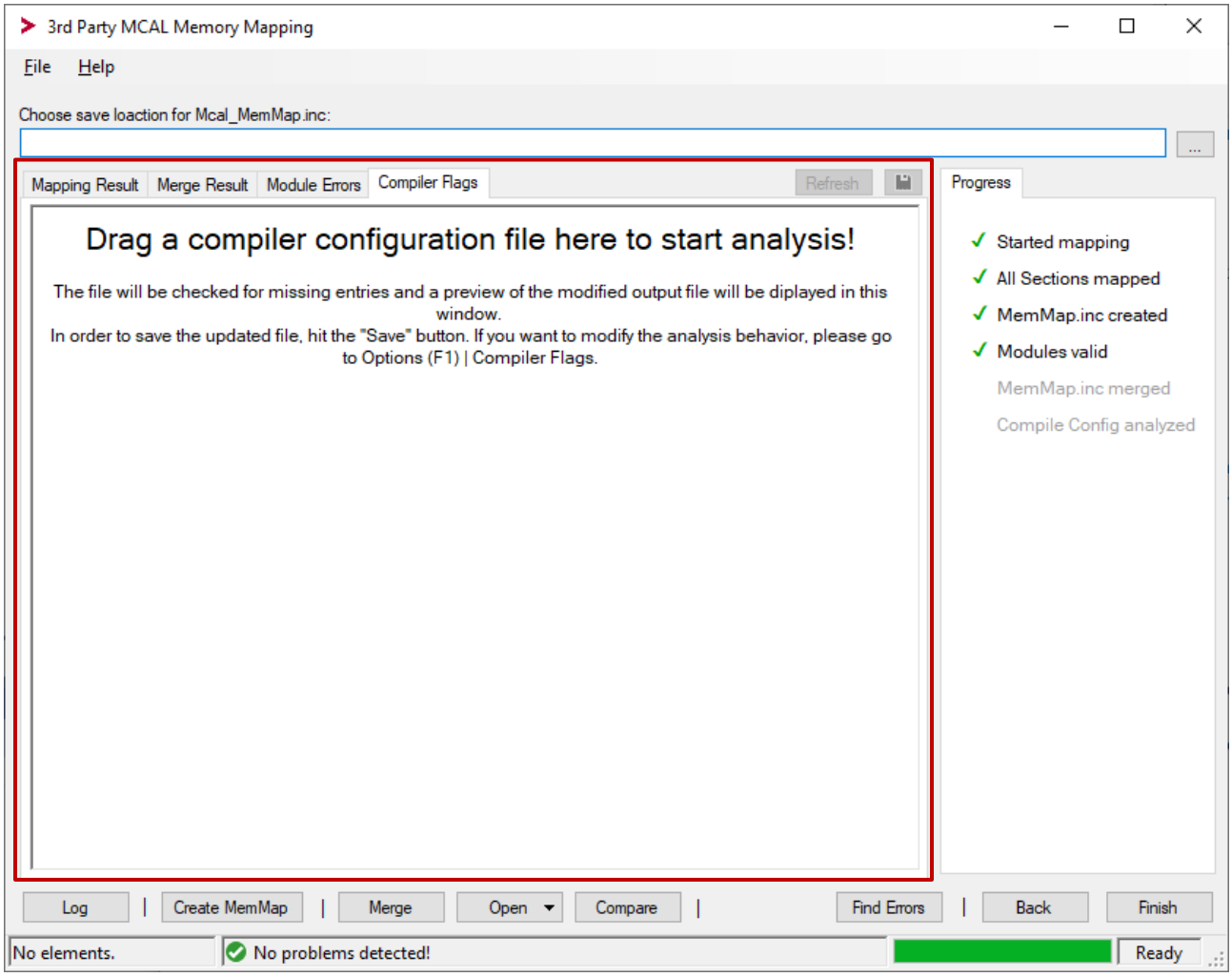


Figure 6-1 Compiler Configuration File Analysis

The compiler configuration file will be analyzed if all specified flags such as `#defines` are available in the file. If not, they are included. The specification of the flags can be done in the main menu. The menu item **File | Options | Compiler Flags** offers the possibility to add new compiler flags or modify the predefined ones:

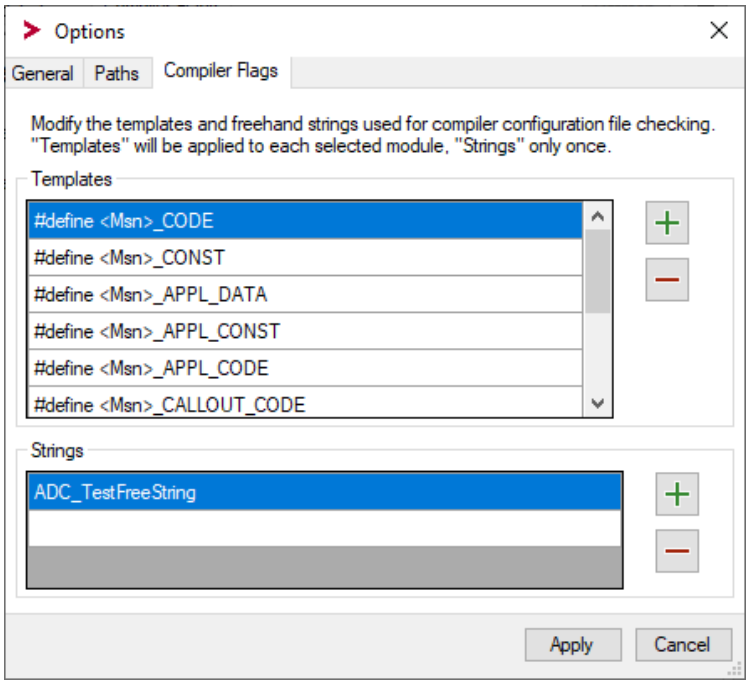


Figure 6-2 Compiler Flag Configuration in Options Dialog

There are two ways to add a new compiler flag: you can use either a template or enter a freehand string. In both cases you first have to click the **[+]** button.

In the template-based entry you just can enter e.g. CODE and the template auto-completes it to `#define <Msn>_CODE` where `<Msn>` stands for module short names like ADC, GPT or MCU. Hence, this define will be searched in compiler configuration file with the module short names of **all activated modules**.

In contrast to this approach, the freehand string you enter will be searched like it is.

Please consider that a change in the compiler flags needs a refresh of the compiler configuration file via the **[Refresh]** button at the upper right edge of the mapper page. This triggers a new analysis of the file if all specified flags are present.

During the refresh all specified compiler flags will be searched in the compiler configuration file. In case that they are missing, the tool inserts them into the file and marks them. The original file will not be changed if you don´t click the **[Save]** button. Please note that the compiler configuration file will be sorted alphabetically and saved afterwards.

© 2022 Vector Informatik GmbH

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
MCAL package	BSW components such as ADC, GPT, MCU
MISC modules	Miscellaneous modules. These are all modules which do not belong to the group of MCAL or CDD modules (from the scope of AUTOSAR)

7.2 Abbreviations

Abbreviation	Description
ADC	Analog Digital Converter
CDD	Complex Device Driver (from scope of AUTOSAR)
GPT	General Purpose Timer
MCAL	Micro Controller Abstraction Layer
3rdPartyMcalMemMap	3rd Party MCAL Memory Mapping tool
MCU	Micro Controller Unit

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com